opflow extensive memory usage for VQE with UCCSD variational form

https://github.com/qiskit-community/qiskit-aqua/issues/1027

ROW 23

opflow extensive memory usage for VQE with UCCSD variational form #1027

Closed #1129

mdsapova opened on Jun 4, 2020

Information

- **Qiskit Aqua version**: 0.7.1
- **Python version**: 3.8.3
- **Operating system**: Ubuntu 18.04
- **RAM**: 8 Gb

What is the current behavior?

I am trying to run VQE for LiH molecule. My current setup is:

- Minimal basis set `sto-3g`
- Backend `statevector_simulator`
- UCCSD variational form
- COBYLA optimizer
- Different size of active space: 6, 8, 10, 12 qubits

I experienced segmentation fault running the 12 qubits configuration (full space of spin orbitals).
It seems like the reason of the segmentation fault is insufficient RAM. The output when running the script with `/usr/bin/time -v`:

```
Command terminated by signal 11
Command being timed: "python LiH_12qubits.py"
User time (seconds): 38.48
System time (seconds): 51.10
Percent of CPU this job got: 105%
Elapsed (wall clock) time (h:mm:ss or m:ss): 1:25.03
Average shared text size (kbytes): 0
Average unshared data size (kbytes): 0
Average stack size (kbytes): 0
Average total size (kbytes): 0
Maximum resident set size (kbytes): 5263392
Average resident set size (kbytes): 0
Major (requiring I/O) page faults: 0
Minor (reclaiming a frame) page faults: 6785484
Voluntary context switches: 0
Involuntary context switches: 0
Swaps: 0
File system inputs: 0
File system outputs: 0
Socket messages sent: 0
Socket messages received: 0
Signals delivered: 0
Page size (bytes): 4096
Exit status: 0
```

Steps to reproduce the problem

Run the gist to reproduce the error. It is based on the example of Qiskit VQE.

What is the expected behavior?

The computation finishes successfully, without an extensive memory consumption.

Suggested solutions

Assignees

jlapeyre

Labels

priority: high type: bug

Type

No type

Projects

No projects

Milestone

No milestone

Relationships

None yet

Development

Be more memory efficient converting SummedOp to MatrixOp
qiskit-community/qiskit-aqua

Notifications Customize

Subscribe

You're not receiving notifications from this thread.

Participants

When I run the 6 or 8 qubits configuration, maximum memory usage is 220 and 460 Mb, respectively. For 10 qubits, the computation finishes as expected, but memory usage increases to 4.6 Gb.

I ran the 10 qubits computation in debugger and found out that a spike in memory consumption happens inside the COBYLA optimizer C code. It happens in a pretty strange way. When the execution enters optimization, memory usage rapidly increases to 4.6 Gb, stays at this level for a short time, and then drops to about 200-400 Mb. For the rest of optimization it does not exceed 400 Mb.

As far as I understand COBYLA algorithm, before the actual iterative process, it evaluates the objective function at `N+1` points (`N` is the number of parameters, 24 for the 10 qubits UCCSD circuit). The memory spike might correspond to the 25 evaluations of the objective function. If it is the underlying reason, can it be avoided (probably at a cost of a slowdown) to significantly reduce the memory consumption?

Additionally, I tried to set COBYLA max iterations to 1 and the spike in memory still occurred. It suggests that indeed something happens during the optimizer initialization.

You can find the code to run the 10 qubits configuration in gist.

woodsp-ibm on Jun 6, 2020 — Member

Hi, in investigating, so far we have tried with COBYLA and SLSQP and get the same failure, also we tried using Aer statevector instead of BasicAer but have the same failure. Now we tried this using 0.6 version of Aqua, in which VQE uses operators that are now legacy in 0.7. In 0.6 Aqua it worked fine. It also works fine in 0.7 if you use Aer qasm_simulator, which by default uses a snapshot instruction in Aer that makes the outcome behave like the statevector. This mode avoids conversion to matrix format of the operator, which is done for evaluation under the statevector simulator, and where we are are beginning to suspect the issue lies in the new operator flow logic that was introduced in 0.7 which is now doing this conversion.

mdsapova on Jun 9, 2020 — Author

@woodsp-ibm thank you for the answer! I checked qasm_simulator and it really worked fine.

woodsp-ibm changed the title ~~COBYLA optimizer extensive memory usage for VQE with UCCSD variational form~~ opflow extensive memory usage for VQE with UCCSD variational form on Jun 11, 2020

woodsp-ibm on Jun 11, 2020 — Member

It appears that the list of paulis (631 of them) is being converted into a list of matrices each having size 4006x4096 (2^12 as there are 12 qubits) of complex128 type. This is 256MB per matrix and consumes a lot of memory. The opflow was done on the basis of lazy evaluation but is leading to excessive memory in this common use case. We will have to look at how things can be improved in this regard.

woodsp-ibm added this to To do in Operators on Jun 11, 2020

woodsp-ibm mentioned this on Jul 9, 2020

memory error: VQE simulation of H2O #1108

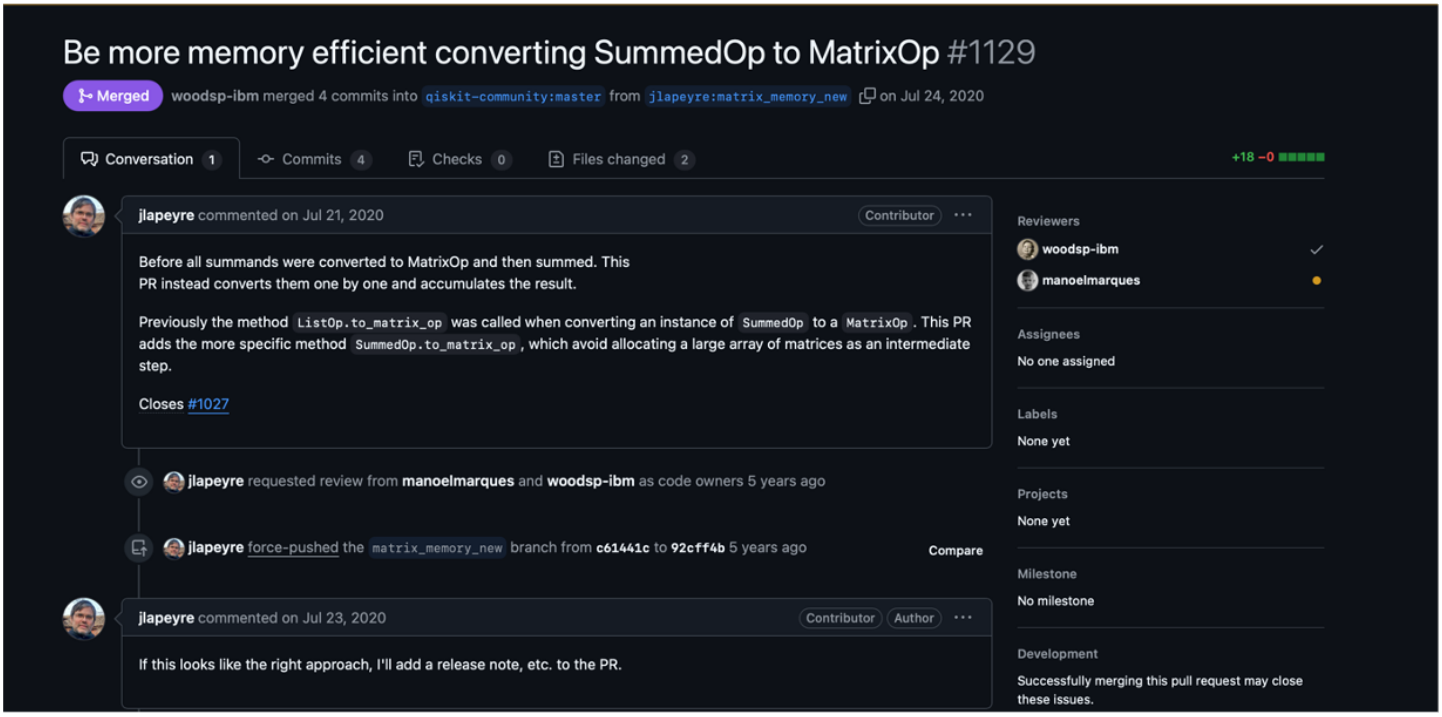
Cryoris added this to the 0.8 milestone on Jul 16, 2020

Cryoris added priority: high type: bug on Jul 16, 2020

woodsp-ibm assigned jlapeyre on Jul 17, 2020

jlapeyre added a commit that references this issue on Jul 21, 2020

`Be more memory efficient converting SummedOp to MatrixOp ...` c61441c


Be more memory efficient converting SummedOp to MatrixOp #1129
Merged
woodsp-ibm merged 4 commits into qiskit-community:master from jlapeyre:matrix_memory_new on Jul 24, 2020
Conversation 1
Commits 4
Checks 0
Files changed 2
+18 −0
jlapeyre commented on Jul 21, 2020
Contributor
Before all summands were converted to MatrixOp and then summed. This PR instead converts them one by one and accumulates the result.
Previously the method ListOp.to_matrix_op was called when converting an instance of SummedOp to a MatrixOp. This PR adds the more specific method SummedOp.to_matrix_op, which avoid allocating a large array of matrices as an intermediate step.
Closes #1027
jlapeyre requested review from manoelmarques and woodsp-ibm as code owners 5 years ago
jlapeyre force-pushed the matrix_memory_new branch from c61441c to 92cff4b 5 years ago
Compare
jlapeyre commented on Jul 23, 2020
Contributor
Author
If this looks like the right approach, I'll add a release note, etc. to the PR.
Reviewers
woodsp-ibm
manoelmarques
Assignees
No one assigned
Labels
None yet
Projects
None yet
Milestone
No milestone
Development
Successfully merging this pull request may close these issues.